



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO
FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN
LABORATORIO DE COMPUTACIÓN GRÁFICA
e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: Bueno Hernández Héctor Daniel

N° de Cuenta: 319233148

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 22/03/2025

CALIFICACIÓN: _____

Antes de comenzar con las actividades, es relevante mencionar que tuve que utilizar un modelo distinto al especificado en mi cuestionario previo para la elaboración de esta práctica debido a que con el modelo escogido originalmente tuve varios problemas.

En primer lugar, este no se encontraba hecho con triángulos, sino con cuadrados. Al momento de intentar triangularlo usando las herramientas de 3ds Max no me fue posible ya que no se le realizaba cambio alguno al modelo. Adicionalmente, no había forma en la que pudiese separar las llantas del modelo del carro debido a que las llantas delanteras y traseras se encontraban unidas entre sí por medio de un cilindro, haciendo que las pares de llantas fuesen un solo modelo en lugar de un modelo para cada llanta.

Por estas razones, decidí trabajar con el modelo de un Hyundai Elantra disponible en el siguiente link: <https://sketchfab.com/3d-models/2024-hyundai-elantra-n-4ba1b1b0eb844e318cc708ada1f2f51f>

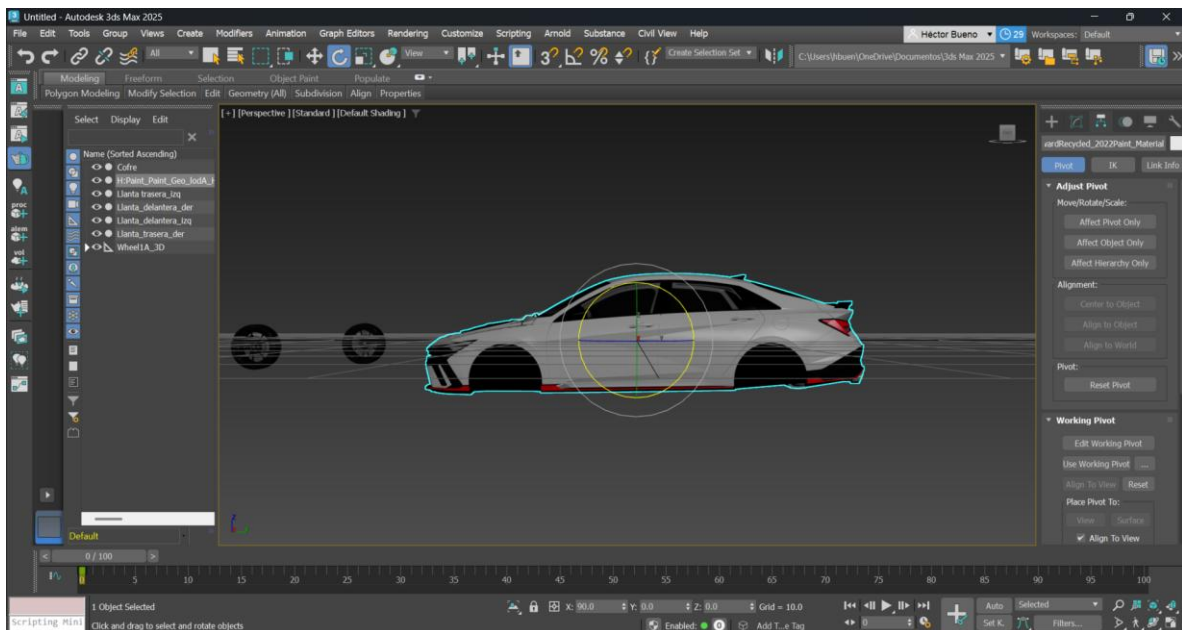


Exportación de los modelos

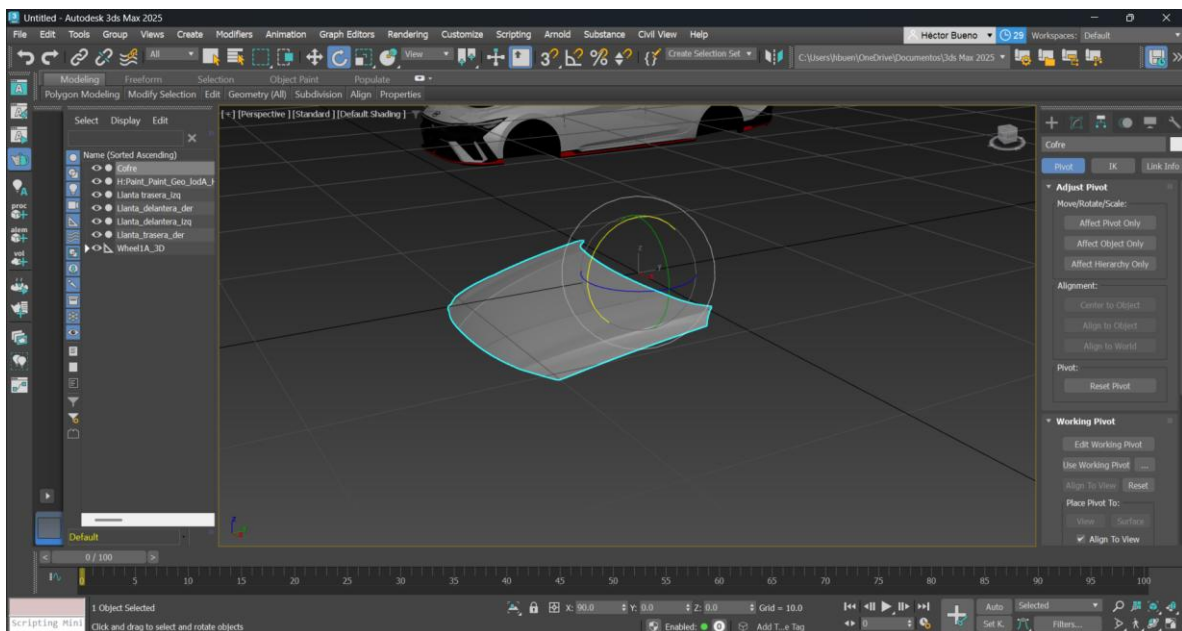
Primero, se solicitó separar los modelos del cofre y cada una de las cuatro llantas del carro por separado para después importarlos en OpenGL siguiendo la estructura del código proporcionado por el profesor para el ejercicio de clase

Para ello, fue necesario hacer uso tanto de la herramienta *dettach* como de *attach*. La herramienta *dettach* se utilizó para separar el cofre del modelo del auto junto con cada una de las llantas. Después, se usó la herramienta *attach* debido a que este se encontraba separado en dos partes. Las llantas también se encontraban separadas en múltiples objetos, por lo que estos se unieron. Así mismo, se unieron todas las partes restantes del auto para tener un solo modelo que incluyera todos los demás objetos existentes.

Finalmente, se modificó el pivote de rotación de cada uno de los modelos y se centraron en el origen para después importarlos en el formato *.obj*. El pivote del carro fue colocado en el centro de este:

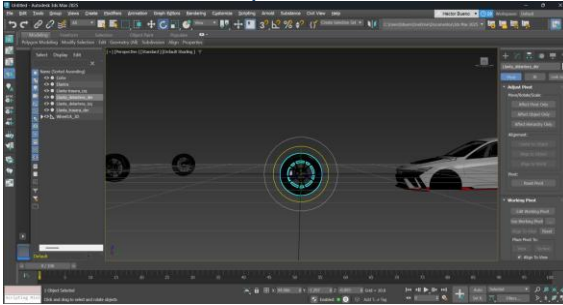


El pivote del cofre se colocó en la parte superior, en medio de los dos extremos para que la rotación se diese a partir de las orillas y no del centro del objeto:



El pivote de las llantas fue colocado en el centro de cada una de ellas:

Llanta delantera izquierda:



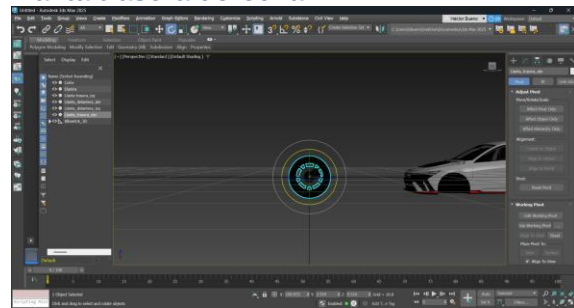
Llanta delantera derecha:



Llanta trasera izquierda:



Llanta trasera derecha:



Importación de los modelos

Una vez que se exportaron todos los modelos requeridos, estos se copiaron a la carpeta *Modelos* del proyecto de Visual Studio para poder ser importados. Ya en el código, primero se declararon todos los objetos de la clase *Model* correspondientes a cada uno de los modelos a importar:

```
Camera camera;  
Model Elantra_M;  
Model Cofre_M;  
Model LlantaDel_Der_M;  
Model LlantaDel_Izq_M;  
Model LlantaTra_Der_M;  
Model LlantaTra_Izq_M;
```

Después, se inicializaron todos los objetos y se les pasan los datos de los modelos contenidos en cada uno haciendo uso de la función *.loadModel*, pasándole como parámetro la ruta del modelo correspondiente junto con el nombre del archivo.

```

Elantra_M = Model();
Elantra_M.LoadModel("Models/elantra.obj");
Cofre_M = Model();
Cofre_M.LoadModel("Models/cofre.obj");
LlantaDel_Der_M = Model();
LlantaDel_Der_M.LoadModel("Models/llanta_delantera_der.obj");
LlantaDel_Izq_M = Model();
LlantaDel_Izq_M.LoadModel("Models/llanta_delantera_izq.obj");
LlantaTra_Der_M = Model();
LlantaTra_Der_M.LoadModel("Models/llanta_trasera_der.obj");
LlantaTra_Izq_M = Model();
LlantaTra_Izq_M.LoadModel("Models/llanta_trasera_izq.obj");

```

Después, se importaron cada uno de los modelos haciendo uso de una jerarquía. Se eligió que el carro fuese el nodo padre, por lo que se utilizó una variable adicional a *modelaux* para almacenar la posición de este nodo llamada *posInicial*. Se le aplicaron transformaciones a los modelos como *rotate* o *translate* para moverlos a su posición indicada, sin embargo, ahora en lugar de llamar los objetos desde una lista, se usan los objetos de *Model* declarados anteriormente. El código generado es el siguiente:

```

//Carro
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(4.0f, 4.1f, 0.0f));
model = glm::translate(model, glm::vec3(0.0f, 0.0f, mainWindow.gettransz())); //Traslación con el teclado
posInicial = model;
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Elantra_M.RenderModel();

//Cofre
model = glm::translate(model, glm::vec3(0.15f, 2.15f, 9.3f));
modelaux = model;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(-1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Cofre_M.RenderModel();
model = modelaux;

//Llanta delantera derecha
model = posInicial;
model = glm::translate(model, glm::vec3(-6.8f, -3.2f, 12.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
LlantaDel_Der_M.RenderModel();
model = modelaux;

//Llanta delantera izquierda
model = posInicial;
model = glm::translate(model, glm::vec3(6.8f, -3.2f, 12.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
LlantaDel_Izq_M.RenderModel();
model = modelaux;

```

```

//Llanta trasera derecha
model = posInicial;
model = glm::translate(model, glm::vec3(-6.8f, -3.2f, -12.1f));
modelaux = model;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
LlantaTra_Der_M.RenderModel();
model = modelaux;

//Llanta trasera izquierda
model = posInicial;
model = glm::translate(model, glm::vec3(6.8f, -3.2f, -12.1f));
modelaux = model;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
LlantaTra_Izq_M.RenderModel();
model = modelaux;

```

Rotación de los modelos

Además de importar los modelos, se solicitó implementar articulaciones para hacer rotar el cofre y cada una de las llantas. Para el cofre la rotación se encuentra limitada a 45° para que esta no de la vuelta completa. Se reutilizaron algunas de las articulaciones declaradas en la práctica anterior, pero modificando las teclas de cada una e implementando los límites de rotación establecidos.

Las teclas para rotar el cofre son **F** y **G**. Cada que se presiona la tecla **F**, se le agregan 5° al ángulo de rotación y se verifica si este ya alcanzó un valor de 45. Si la condición se cumple, el ángulo se iguala a 45° para ya no permitir que este siga aumentando. Por el otro lado, Cada que se presiona la tecla **G**, se le restan 5° al ángulo de rotación y se verifica si este ya alcanzó un valor de 0. Si la condición se cumple, el ángulo se iguala a 0° para ya no permitir que este siga disminuyendo.

```

//teclas para la rotación del cofre
if (key == GLFW_KEY_F)
{
    theWindow->articulacion1 += 5.0;
    if (theWindow->articulacion1 > 45.0) {
        theWindow->articulacion1 = 45.0;
    }
}
if (key == GLFW_KEY_G)
{
    theWindow->articulacion1 -= 5.0;
    if (theWindow->articulacion1 < -0.0) {
        theWindow->articulacion1 = -0;
    }
}

```


En el caso las llantas, no se estableció un límite a la rotación ya que las llantas deben dar vueltas completas tanto para delante como para atrás. Cuando se presiona la tecla **L** se le agregan 15° a la rotación y cuando se presiona la tecla **K** se le restan 15° a la rotación. La rotación aumenta más rápido en comparación con el aumento de rotación del cofre para simular una mayor velocidad de giro en las llantas. En esta ocasión solamente se está haciendo uso de la *articulación2* ya que todas las llantas giran al mismo tiempo al presionar las teclas antes mencionadas.

```
//teclas para la rotación de las llantas
if (key == GLFW_KEY_L)
{
    theWindow->articulacion2 += 15.0;
}
if (key == GLFW_KEY_K)
{
    theWindow->articulacion2 -= 15.0;
}
```

Traslación con el teclado

Por último, también se solicitó agregar traslación al carro de tal forma que este avance y retroceda al presionar una tecla de forma similar a la rotación con el teclado. Para ello, se declararon tres variables dentro de *Window.h* llamadas *transx*, *transy* y *transz*. En esta ocasión solamente se usará *transz*, pero las demás podrán usarse en otros proyectos si se desea.

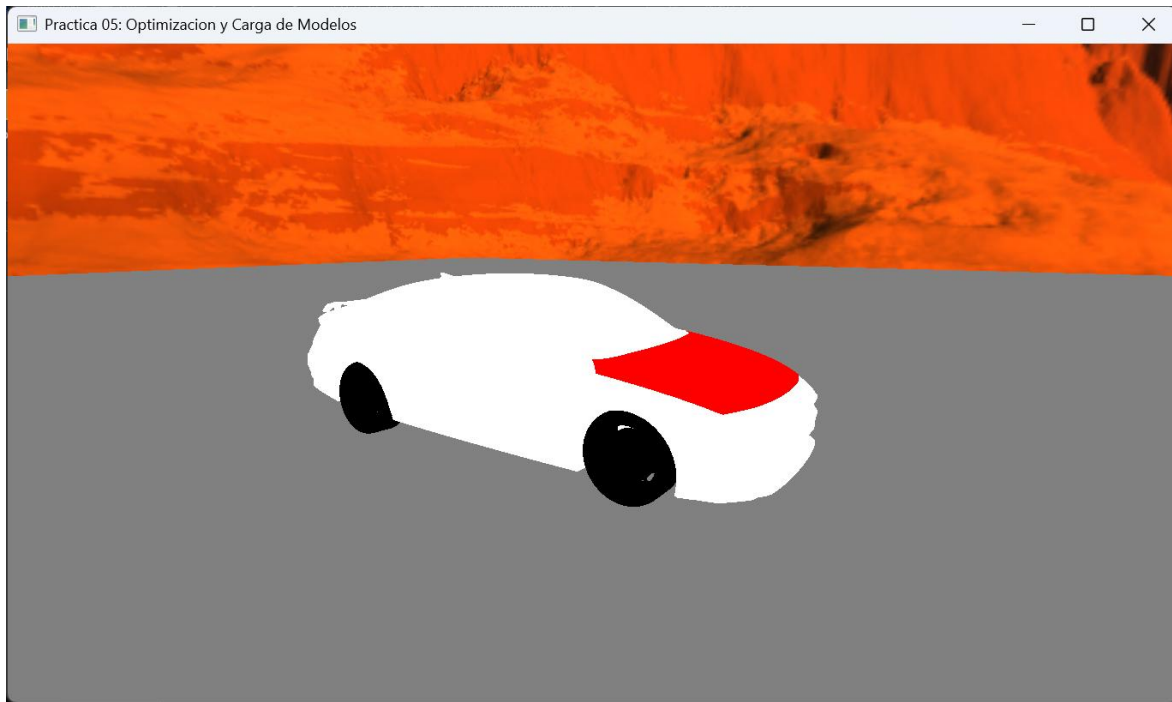
Después, se generó el siguiente bloque de código donde se le agrega 5 unidades *transz* al presionar la tecla **P** y se le restan 5 unidades al presionar la tecla **O**:

```
if (key == GLFW_KEY_P)
{
    theWindow->transz += 5.0; //avanzar sobre el eje z 10 unidades
}
if (key == GLFW_KEY_O)
{
    theWindow->transz -= 5.0; //retroceder sobre el eje z 10 unidades
}
```

Al momento de declarar la traslación, esto se hizo de forma normal pero sustituyendo el valor correspondiente al eje Z con la función, *mainWindow.gettransz()*. El valor regresado por la función, es decir, el valor de *transz*, será la distancia trasladada en Z.

```
//Carro
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(4.0f, 4.1f, 0.0f));
model = glm::translate(model, glm::vec3(0.0f, 0.0f, mainWindow.gettransz())); //Traslación con el teclado
posInicial = model;
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Elantra_M.RenderModel();
```

La ejecución del código ya con todos los modelos importados y colocados en su posición ve de la siguiente manera:



Problemas presentados

No presenté ningún problema al momento de realizar la práctica ya que el procedimiento de esta fue bastante similar al llevado a cabo en el ejercicio de clase. Si bien llegue a presentar problemas con el modelo del carro que había escogido inicialmente, esto fue resuelto al cambiar el modelo ya que el nuevo si contaba con las especificaciones necesarias. Adicionalmente, no conté con ningún problema al momento de ejecutar el código.

Conclusión

Considero que la dificultad de la práctica fue bastante adecuada ya que me permitió seguir familiarizándome con el manejo del programa 3ds Max para la manipulación de modelos 3D. Asimismo, esta práctica me sirvió para seguir practicando el manejo de transformaciones de objetos como rotaciones y traslaciones, tanto con el teclado como con valores ya establecidos.

Creo que la explicación del profesor fue bastante adecuada ya que nos guio paso a paso en cómo utilizar 3ds Max junto con las herramientas que usaríamos en esta práctica. Además, los comentarios contenidos en el código proporcionado también fueron de ayuda para saber en qué partes era necesario realizar las modificaciones pertinentes y esto fue bastante útil al momento de importar y modificar los modelos.

Bibliografia

LearnOpenGL - Transformations. (s. f.). <https://learnopengl.com/Getting-started/Transformations>

Sketchfab. (s. f.-a). 2024 Hyundai Elantra N - Download Free 3D model by Ddiaz Design (@ddiaz-design). <https://sketchfab.com/3d-models/2024-hyundai-elantra-n-4ba1b1b0eb844e318cc708ada1f2f51f>